

Benchmark Report: pyflate

A small link budget, a compressed payload, and a lot of work between the bits

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Repository: github.com/matanco64/HWSW_Project

Somewhere between a spacecraft and a ground station there is a link budget measured in kilobits. Compression makes the most of it; decompression turns the arriving bits back into useful data. This benchmark gives that quiet half of the job to Python: read a bit stream, recover symbols, and undo several transformations. Its Huffman matcher looks like the obvious place to start. Measuring it led us to improvements across the whole decoding pipeline, then a Rust extension that executes the symbol decoder through one call per block.

1. Overview: workload and result

Despite its name, the measured pyflate workload is **bzip2**, not DEFLATE. Each iteration decompresses `interpreter.tar.bz2` from **67,562 bytes to 399,360 bytes**; the harness checks the original MD5 outside the timer. The Python tier uses only the standard library (`re` for the final run-length stage, `collections.Counter` for the Burrows-Wheeler transform (BWT) histogram, `hashlib` for MD5) plus `pyperf` for timing; the *native Rust* tier adds our compiled Rust crate bound through `PyO3`, as opposed to Python run by the interpreter. *Original* means the benchmark exactly as shipped in `pyperformance`, unmodified. No native decompression library is called. Its main structures are an integer bit buffer, Huffman tables, a move-to-front (MTF) alphabet, BWT index vectors and output buffers: concretely a Python `int` bit window, `list` lookup tables packing (`symbol`, `length`) into one integer, a `list` MTF alphabet and traversal vector, and `bytearray` outputs.

One block, six Huffman tables, 148,271 symbols. The decoder switches tables every 50 symbols. Huffman decoding, move-to-front and RUNA/RUNB expansion (bzip2's two run-length symbols) produce a 336,184-byte L-vector, the last column of the BWT matrix. Inverse BWT and the final run-length expansion produce the 399,360-byte output. We call that last stage **RLE4**: four equal bytes followed by a count byte `k` stand for `4 + k` copies.

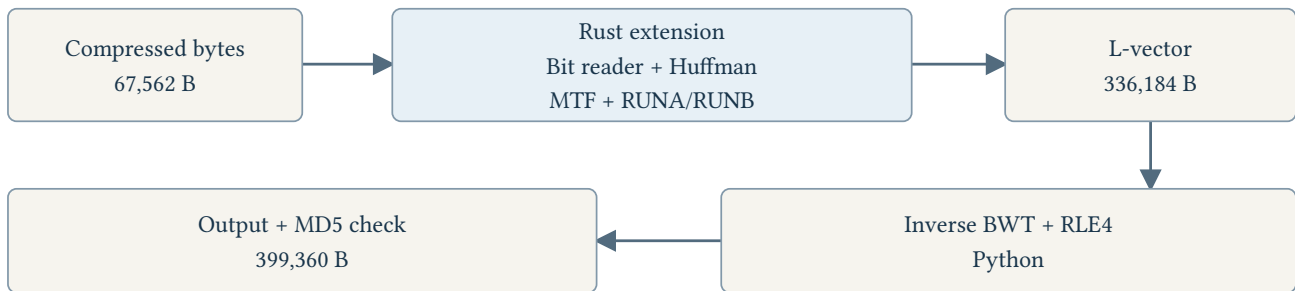


Figure 1: The bzip2 path. RUNA/RUNB and move-to-front are handled together in the symbol loop. The Rust kernel replaces that boundary in one call per block, and it is the same boundary the two accelerators implement.

Configuration	Mean ± SD	vs original
Original Python	1,123.49 ± 10.99 ms	1.00×
Optimized Python	281.16 ± 3.05 ms	4.00×
Python + Rust extension	170.01 ± 2.30 ms	6.61×

Our final software path is 6.61× faster than the original: an 84.9% runtime reduction. Python improvements provide 4.00×; over the same benchmark's Python back end, the Rust symbol decoder adds 1.67× (Section 4; its baseline is 283.88 ms, not 281.16 ms, so the two ratios do not multiply exactly). Both tiers exceed the course's 7% improvement requirement and match `bz2.decompress` byte for byte, passing the unchanged MD5 check. Rust handles the stage labeled Rust extension above; block headers, inverse BWT and RLE4 remain in Python.

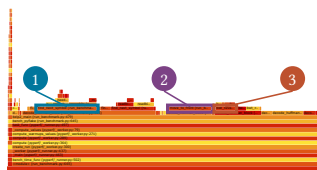
Measurement scope: Course QEMU/KVM virtual machine (VM), Ubuntu 22.04, release CPython 3.10.12; 120 values per configuration, pinned to guest CPU 0, from revision 2c8c754 through the documented scripts and its own Rust build. Source: results/vm_canonical_20260910_2c8c754/suite/ (Appendix A7). Development profiles and ablations are labeled separately. Later commits trimmed comments, moved the BWT histogram to collections.Counter and lowered the Rust code-length cap from 23 to 20; they pass the same byte-exact checks but were not re-timed.

Appendix A5 gives the timing distributions and worker-clustering analysis. The repeated measurements support a large improvement on this input; one bzip2 fixture does not establish the same speedup for every file, compression setting or interpreter.

2. Initial analysis: profiling the whole decoding pipeline

Two profilers answer two different questions. The brief's `perf record -F 999` on `python3-dbg` records interpreter C frames (`_PyEval_EvalFrameDefault` 99.50% inclusive; results/perf_report_pyflate_stock.txt): it cannot name a Python function, but it shows what the interpreter spends its time on, and the table at the end of this section reads it before and after the optimizations. `py-spy` on release CPython (Python frames), rendered with `FlameGraph`, names the functions. The original Python-frame profile highlights Huffman decoding and bit-buffer work, with move-to-front and inverse BWT alongside them. Development instrumentation found a mean of about five entries visited by the symbol matcher, despite its table capacity of 258 (`dev/pyflate/instrument.py`; `dev/pyflate/FINDINGS.md`: mean scan length 5.0 entries). A lookup table can remove this scan, but the measured scan is short; per-symbol interpreter work and the later transformations also deserve attention.

Full recorded flame graph

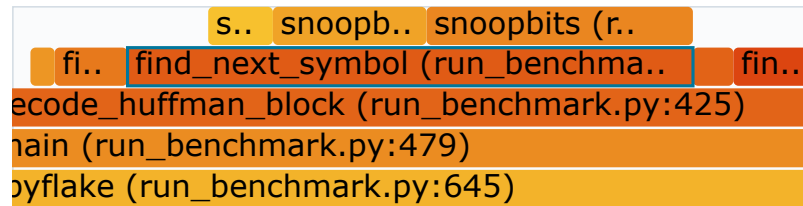


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
`py-spy` emits one frame per
source line, so a function is
many boxes; shares are totals.

Highlighted regions - enlarged with surrounding frames

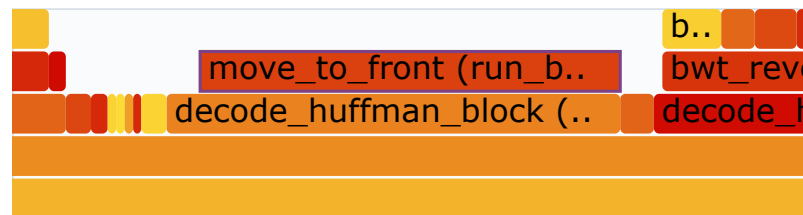
1. Huffman matcher

`find_next_symbol` | 40.87% inclusive, across 7 per-line frames; widest 18.82% outlined



2. Alphabet updates

`move_to_front` | 13.44% inclusive



3. Inverse BWT

`bwt_reverse` | 14.52% inclusive, across 5 per-line frames; widest 6.18% outlined

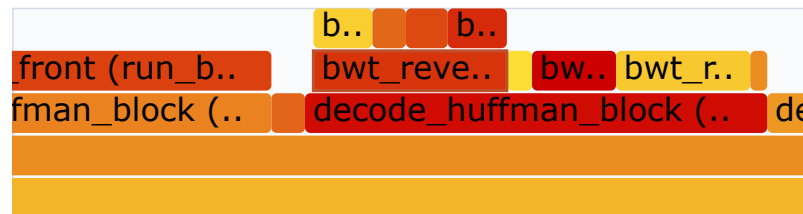


Figure 2: Full original Python-frame flame graph (372 samples), retaining startup and harness context. Numbered outlines identify exactly the call paths enlarged on the right. Inclusive percentages use the original whole-profile denominator; nested shares overlap. Original: results/pyspy_pyflate_stock_full.svg.

Function (py-spy; 372 / 122 samples)	Original self	Original incl.	Opt. self	Opt. incl.
decode_huffman_block	26.08%	97.31%	1.64%	82.79%
_decode_symbols_python	–	–	32.79%	32.79%
find_next_symbol	12.63%	40.86%	–	–
move_to_front	13.44%	13.44%	0.82%	0.82%
snoopbits	9.41%	16.13%	–	–
readbits	9.95%	13.44%	2.46%	2.46%
_mask	5.91%	5.91%	–	–
bwt_reverse	8.33%	14.52%	13.11%	38.52%
bwt_transform	6.18%	6.18%	25.41%	25.41%
rle4_expand	–	–	5.74%	5.74%

Self excludes callees; **inclusive** includes them. Each profile has its own denominator, so percentages locate remaining work rather than measure absolute speedup. The optimized symbol loop incorporates the old matcher, bit reader and most MTF handling; residual `readbits` / `move_to_front` calls parse header selectors. Figure labels aggregate a function's per-line frames, matching the table within rounding; outlines mark one frame. These are 372- and 122-sample profiles: a 1% cell is one to four samples, so the tables locate work; they do not resolve differences below a few percent.

Symbol decoding is the largest optimized function (32.79% self), while whole inverse BWT accounts for 38.52% inclusive. Both matter for the next offload decision. These sparse profiles identify candidates; Section 4 explains why they cannot supply a precise Amdahl fraction for the separately measured native timing pair.

What the interpreter is doing: the C-frame profiles (`perf`, 999 Hz cpu-clock, debug build) of the original and the optimized decoder, self time per symbol family. The grouped rows sum self time over the regexes in `results/profile_functions.txt` (`report/summarize_profiles.py`).

C-frame profile, self time	Original	Optimized
_PyEval_EvalFrameDefault (bytecode dispatch)	23.10%	20.82%
_PyMem_DebugCheckAddress (debug allocator)	3.83%	6.42%
read_size_t (allocator bookkeeping)	3.06%	3.21%
call_function	2.44%	0.92%
list_dealloc	2.29%	0.56%
list_ass_slice (the original MTF update)	2.05%	0.25%
list_item	1.38%	4.55%
binary_op1 (generic arithmetic dispatch)	1.57%	1.48%
sre_ucs1_match (regex, RLE4)	–	2.12%
Allocation and object lifetime, all symbols	23.37%	30.95%
List access and index conversion, all symbols	9.93%	9.91%
Attribute lookup and call machinery, all symbols	9.45%	2.13%
Integer arithmetic (the bit window), all symbols	6.30%	8.04%

Each profile has its own denominator. The original decoder spends a quarter of its time allocating and freeing objects (a per-byte int and list slice for every symbol) and almost a tenth in call machinery; the optimizations remove the call and slicing families (`call_function` 2.44% → 0.92%, `list_ass_slice` 2.05% → 0.25%, calls 9.45% → 2.13%) and leave a flatter loop whose remaining cost is the interpreter itself, int arithmetic on the bit window and the allocator. Those three do not shrink by rewriting Python; they are why the native decoder in Section 4 pays off. The debug build overstates the allocator share (`_PyMem_DebugCheckAddress` exists only in `python3-dbg`), so these are shapes to read, not release-build costs.

3. Optimizations: changes that shipped

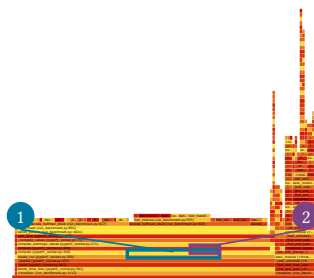
- **Reduce per-byte overhead:** read the compressed input once, refill the bit window in chunks, bind hot values to locals and accumulate bytes without allocating an object per byte.
- **Decode canonical Huffman codes:** use length-indexed tables and a primary lookup table with a fallback for longer codes. Table construction is amortized over the block.
- **Improve the back end:** replace comparison sorting with counting-sort BWT construction; reverse the MTF list so updates move $O(\text{rank})$ entries; use regex-assisted RLE4 expansion to reduce Python loop iterations. The byte scan/output work is still linear overall.

Component reverted from T3	VM trial 1	VM trial 2	Dev, 3.12
None (full T3 runtime)	271.2 ms	273.9 ms	175.5 ms
Regex-assisted RLE4	+100.6 ms	+98.7 ms	+32.3 ms
Primary Huffman lookup	+52.6 ms	+49.4 ms	+10.1 ms
Counting-sort BWT	+20.8 ms	+17.2 ms	+38.6 ms
All reverted (T1 only)	+393.6 ms	+397.2 ms	+200.6 ms

Ablation scope: T3 is the last tier of our development ladder (T0 original, T1 per-byte fixes, T2 canonical decode, T3 the shipped decoder). Each row reverts one optimization in the optimized Python decoder (the lookup row falls back to canonical length-stepping, not the original scan) (dev/pyflate/ablate.py, which drives the T3 module directly rather than the pyperf harness) and reports the added time, best of seven interleaved decompressions. Costs need not add. The two VM trials are separate runs on the course VM, release CPython 3.10.12, pinned to one guest CPU; the last column is the earlier development run on Windows / CPython 3.12.6 (dev/pyflate/FINDINGS.md §2b).

The measured platform reverses part of the development ranking. On the VM, regex-assisted RLE4 is worth about 100 ms and the primary Huffman lookup about 50 ms – both more than counting-sort BWT at about 20 ms – and the two trials agree within 4 ms on every row. The development run on 3.12 had ranked BWT first and the Huffman table last, so the ordering depends on the interpreter and platform, not only on the algorithms. What survives on both is that no single change dominates: the pipeline, not the matcher alone, had to be optimized. Raw output: results/vm_rerun_20260910_3697a63/.

Full recorded flame graph

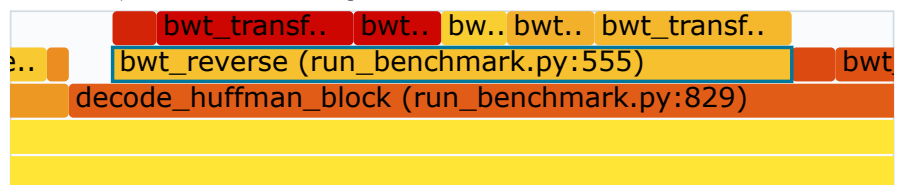


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
py-spy emits one frame per
source line, so a function is
many boxes; shares are totals.

Highlighted regions - enlarged with surrounding frames

1. Inverse BWT

bwt_reverse | 38.53% inclusive, across 4 per-line frames; widest 25.41% outlined



2. Index-table construction

bwt_transform | 25.42% inclusive, across 6 per-line frames; widest 7.38% outlined

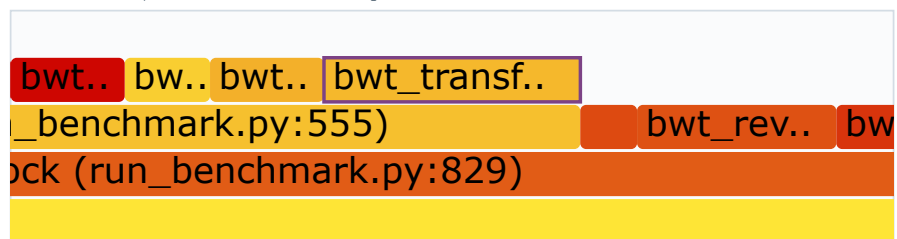


Figure 3: Full optimized **Python** profile (122 samples), before native offload. Inverse BWT and its index-table construction remain visible beside `_decode_symbols_python`, the single symbol-decode loop that replaced the original matcher and bit-reader frames. Original: results/pyspy_pyflate_opt_full.svg.

Worked examples of the shipped transformations

Before/after, removing the per-symbol search: The original matcher visits Huffman entries and calls the bit reader for candidate lengths. The optimized loop peeks once into a flat table; a nonzero entry packs the

symbol and consumed length. These excerpts summarize the control flow, omitting refill, table switches and output handling.

Original matcher (schematic)

```
for entry in table:
    code = field.snoopbits(entry.bits)
    if code == entry.code:
        field.readbits(entry.bits)
        return entry.symbol
```

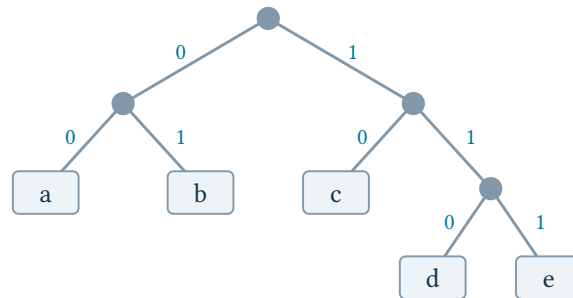
Optimized lookup (abbreviated)

```
v = tbl[peek(pb)]
if v:
    consume(v & 31)
    symbol = v >> 5
else:
    # Extend bits using limit/base/perm.
    symbol = canonical_fallback()
```

peek, consume and canonical_fallback are explanatory names; the shipped loop performs these operations on local integers. Source: build_huffman_table and _decode_symbols_python in benchmarks/bm_pyflate/run_benchmark.py.

Example codes

```
a = 00
b = 01
c = 10
d = 110
e = 111
```



Canonical ranges

len 2: 00..10

len 3: 110..111

Match a length range,
then index the symbol.

Example: 110 decodes to d.

Figure 4: Illustrative canonical Huffman codes. Codes of equal length are consecutive; length-indexed ranges identify the symbol without scanning individual candidates. The optimized implementation adds a primary lookup for short codes.

A four-entry lookup, step by step: Use the illustrated code lengths [2, 2, 2, 3, 3] and temporarily choose $pb = 2$ (the shipped default is 11, capped by the longest code). The primary entries for 00, 01, 10, 11 are (a, 2), (b, 2), (c, 2), and fallback. On input 01110, peek 01, output b and consume two bits; 110 remains. Peek 11: its entry is zero, so extend to three bits. Now code 6 lies at or below $limit[3] = 7$; $base[3] = 3$, hence $perm[6 - 3] = perm[3] = d$. Consume three bits. Thus the stream decodes to b,d without searching individual symbols. With a three-bit primary table, 00x, 01x and 10x each duplicate a two-bit code into two slots; the stored length still consumes only two bits, preserving the next symbol's first bit.

Counting-sort BWT construction: For $L = \text{banana}$, counts are a:3, b:1, n:2. Prefix sums place their buckets at offsets 0, 3 and 4. Scanning L left to right stores each original index in the next slot of its bucket, giving $T = [1, 3, 5, 0, 2, 4]$. Equal symbols keep their order of occurrence. The original path sorts L and searches for bucket starts; the optimized path counts 256 possible byte values and fills T in $O(n + 256)$ work. This constructs the traversal table; inverse BWT still follows $end = T[end]$ once per output byte.

Reversed move-to-front: Start with logical order [a, b, c, d], rank 2 selecting c. The original slicing update produces [c, a, b, d] by rebuilding the list. The optimized physical list is reversed, [d, c, b, a]. The non-run symbol is $r = rank + 1 = 3$; $pop(-3)$ returns c, then $append(c)$ leaves [d, b, a, c], the reverse of the same logical answer. The pop shifts only two trailing entries and append places c at the logical front. This preserves the operation while moving $O(rank)$ entries instead of rebuilding the whole alphabet.

RLE4 remains a separate final stage. Four equal bytes followed by count k represent $4+k$ copies. For example, [A, A, A, A, 2, B] expands to six A bytes and one B. The implementation's regex locates runs in C and bulk byte operations construct the repeated span; it does not change the bzip2 representation or the linear size of the output.

4. Performance comparison: native execution, what improves and what remains

Our Rust/PyO3 BlockDecoder handles the bit reader, Huffman decode, MTF and RUNA/RUNB expansion. Header parsing, inverse BWT, RLE4 and MD5 remain in Python. One block-level call returns the L-vector and the ending bit position; checking both catches output errors and stream desynchronization.

Same benchmark file	Mean $\pm$ SD	vs Python
Python backend	283.88 $\pm$ 3.34 ms	1.00×
Native decode backend (Python + Rust extension)	170.01 $\pm$ 2.30 ms	1.67×

The native row is the same measurement as the Python + Rust row on page 1. The Python row runs the same benchmark file with its Python back end directly under pyperf (the timing library), not through pyperformance (the suite runner that wraps it), so it differs slightly from the optimized Python row on page 1; both cover the complete benchmark. `HWSW_BACKEND=python|native|auto` selects the implementation; native mode requires the extension, while auto mode falls back. The benchmark forwards the variable to pyperf's workers itself, so a scrubbed worker environment cannot quietly change which back end is measured.

The submitted Rust source was rebuilt on the VM. All eleven crate tests pass there, together with seven blocks across five Python/bz2 fixtures, including exact ending bit positions. The eleventh is a property test that decodes random Kraft-complete (no unused code space) Huffman tables against an independently written canonical encoder; it was added after the timed revision, so the timed run's log records the ten that existed then, and `results/vm_verify_20260921_f407567/` records all eleven on the same VM at the submitted revision. A dispatch check confirmed that native and auto load the new extension and call it once per block.

Matched-work CPU counters

These are medians of three warm-loop VM runs, each decoding the input 16 times, divided by 16. Both backends produced the same output digest. They are a separate experiment from the rigorous timing table above, with counters enabled only around the timed loop (Appendix A2).

Metric per complete decode	Python backend	Python + Rust extension
Elapsed time	288.68 ms	174.44 ms
Instructions	1,947.94 M	1,171.65 M
Cycles	675.92 M	404.24 M
IPC (instructions per cycle)	2.88	2.90
Branches	349.43 M	207.30 M
Branch misses (rate)	544,260 (0.156%)	301,458 (0.145%)
Generic cache references	1.749 M	1.686 M
Generic cache misses (rate)	69,322 (3.964%)	52,842 (3.133%)

Native decoding reduces instructions by **39.9%**, cycles by **40.2%** and branch misses by **44.6%**, while IPC changes little. The approximately 1.65× speedup in this experiment comes mainly from removing interpreter work. Generic cache misses fall **23.8%** in this capture (an earlier capture showed about 2%, so we draw no conclusion from the size of that reduction): the remaining Python BWT/RLE4 pipeline still processes the whole block. Aggregate counters cannot attribute misses to a particular stage. L1 events are omitted because an earlier capture returned invalid load counts.

Offload limits and the next target

Amdahl's law gives $S = 1 / ((1 - f) + f/s)$ for kernel speedup s and offloaded fraction f of **optimized Python** runtime. Even infinite symbol-decoder speed leaves BWT and RLE4. The sampled 32.79% decoder share is a hotspot locator, not a calibrated value of f for the native timing pair: it would imply a maximum 1.49× gain, below the measured 1.67×. Under the simple unchanged-residual model, 1.67× requires at least 40.1% offloadable work. Separate sampling, setup inside the decoder, and differences between measured runs prevent identifying f from this profile alone. Section 5's derived 117 ms Python symbol stage, about 41% of 283.88 ms, is consistent with that bound. Matched phase timing is needed to explain the ratio quantitatively; the end-to-end measurement itself remains the evidence for the speedup.

Unlike nbod's native integration loop, this offload leaves over a billion instructions per decode in a largely Python pipeline. The next target is the **whole inverse BWT**, including table construction, not just its dependent traversal. Earlier VM phase timings measured **137.7 ms for whole inverse BWT** and **47.8 ms for traversal alone** (Appendix A3). The first includes construction and traversal; the two numbers must not

be added. A native port and working-set sweep would test whether memory latency then becomes limiting; the present data do not establish that.

5. Hardware acceleration proposal: implementation and estimates

bzip2's symbol stage is two different jobs back to back. *Huffman decoding*: the compressed input is a stream of variable-length codes, each matched against the block's code tables to recover a symbol. `huffman_engine` does this, comparing the next bits against every code length in parallel, one symbol per cycle. *Move-to-front and run expansion*: each symbol is either a rank into a list of recently used bytes (output that byte, move it to the front) or part of a run count to expand. `mtf_cam` does this with a 256-entry byte list held in a parallel shift register, read by rank and re-ordered in one cycle. Chained on chip they produce the same L-vector as the Rust kernel, over the same boundary. Software keeps block headers, inverse BWT, RLE4 and MD5: per block it parses the header, writes the code lengths and parameters through the register window, starts both modules and collects the L-vector, the same single call per block that the Rust kernel replaces. A register-level driver model implements this protocol; it is not a deployed device driver.

Why two modules: the two jobs want different hardware. Huffman decoding is table-driven and its cost is storage (table and register storage is 95% of `huffman_engine`); move-to-front is a wide list update whose risk is the output rate (246,347 of the 336,184 L-vector bytes, 73.3%, come from run groups; `hw/mtf_cam/docs/prd.md` §1). Separate modules are verified against separate golden models and sized by separate trade-offs, and the Huffman block stays reusable (it also implements DEFLATE). They are chained on chip so the 148,271 intermediate symbols never cross to software; leaving move-to-front in software would have kept its 13.44% share of the original runtime on the CPU.

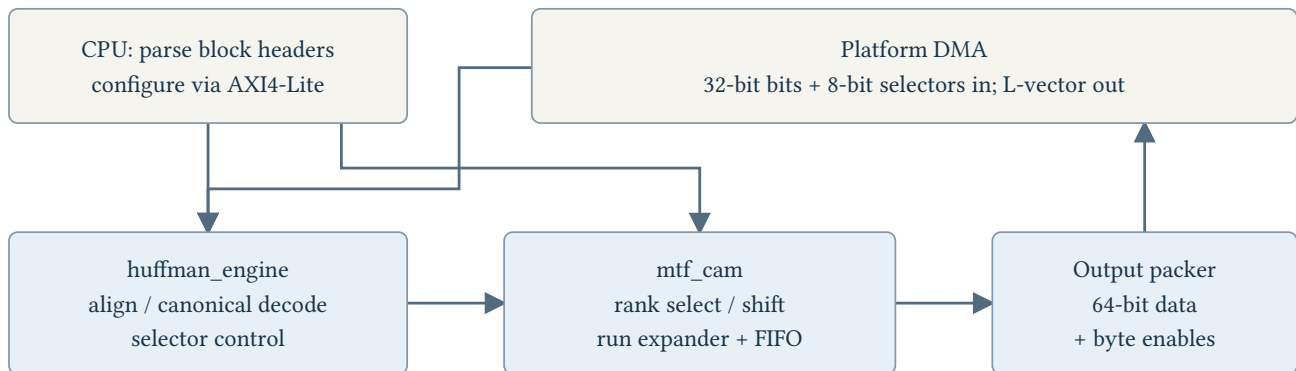


Figure 5: Block diagram: AXI4-Lite configures both modules; AXI4-Stream carries data. The symbol stream stays on chip. Platform DMA returns the L-vector to software.

Block diagram in words: CPU driver → AXI4-Lite → registers of `huffman_engine` (code lengths) and `mtf_cam` (used-byte map). Compressed bits and table selectors → AXI4-Stream → `huffman_engine` → 32-bit symbol beats, on chip → `mtf_cam` (move-to-front and run expansion) → AXI4-Stream bytes → platform DMA → the L-vector in memory. Each module raises its own interrupt. Full drawings: `hw/huffman_engine/docs/block_diagram.svg`, `hw/mtf_cam/docs/block_diagram.svg`; chain wrapper `hw/pyflate_accel/rtl/pyflate_accel.sv`.

Huffman engine

A bit aligner feeds a canonical-code comparator/lookup datapath; a selector controller changes the active table every 50 symbols. Inputs are a 32-bit compressed stream and an 8-bit selector stream; outputs are 32-bit beats carrying a 9-bit symbol plus type fields. A 32-bit AXI4-Lite window holds controls, `START_BIT`, lengths and counters, for six tables and bzip2's 20-bit maximum code length. It decodes the benchmark block in **149,276 cycles** for 148,271 symbols, so **1.0068 cycles per symbol** including table construction and switches, and is trace-exact against the golden model over every beat.

MTF and run expansion

The input is a **rank**, so the decoder indexes the alphabet, shifts the preceding entries and expands RUNA/RUNB groups through a buffered output packer at eight bytes wide. Inputs are the decoder's 32-bit symbol beats; outputs are 64-bit beats of L-vector bytes with an 8-bit byte-valid mask, and a 32-bit AXI4-Lite window holds control, status and counters. It sustains **1.0686 cycles per symbol**, and the move-to-front list is 68% of its area.

Simulated and synthesized cost

How these numbers were produced: each accelerator is written in SystemVerilog and simulated cycle by cycle (Verilator, cross-checked with Icarus) inside a Python testbench that feeds it the real benchmark block and compares every output against a *golden model*, a Python reference of the same stage. Cycle counts, test results and coverage come from these runs. The design is then *synthesized*: Yosys translates it into a netlist of standard logic cells from *sky130*, the open-source SkyWater 130 nm process, which gives cell count and area. Finally OpenLane *places* the cells, builds the clock tree and runs *static timing analysis*, which sums gate and wire delays along every register-to-register path; the slowest path sets the maximum clock. We stop after clock-tree synthesis (post-CTS): cells and the clock network are placed but signal wires are not routed, so the frequency is an estimate, not silicon. The toolchain is entirely open source, so every number can be regenerated from the repository.

sky130, Yosys + OpenLane	huffman_engine	mtf_cam
Cells (synthesis)	151,058	18,814
Area (synthesis)	1.634 mm ²	0.187 mm ²
Largest area share	tables + registers, 95%	move-to-front list, 68%
Timing-derived frequency (post-CTS)	≈ 39.9 MHz	≈ 37.5 MHz
Power estimate (post-CTS)	≈ 283 mW	≈ 10.2 mW
Cycles per symbol (simulation)	1.0068	1.0686
Directed + random tests	17 / 17	16 / 16
Line / toggle coverage	90.4% / 90.3%	92.0% / 93.8%

Evidence levels: both frequencies come from post-CTS static timing at the typical corner: 39.9 MHz for Huffman (40 ns constraint, +14.94 ns worst setup slack; a tighter 27 ns run also meets timing and gives 39.5 MHz) and 37.5 MHz for MTF (27 ns constraint, +0.31 ns worst setup slack). Each module's 50 MHz run did not meet timing; MTF's 20 ns run failed by 6.6 ns, and the 37.6 MHz it back-computes agrees with the met run within 0.4%. The quoted frequencies are from runs that met their constraint. They are estimates, not demonstrated silicon operating frequencies; routing may change either, and neither completed 50 MHz sign-off or produced a final layout (GDS). The power figures are tool estimates using default switching activity at each run's own clock constraint (40 ns for Huffman, 27 ns for MTF), so they are not comparable with each other, are not workload power, and cannot support energy savings. Huffman is 8.7× the area of MTF because its code tables are flops, and its 283 mW has the same cause: the estimate toggles the 1,728-entry symbol table and the 288-word length window (54% of its cells are flops) at default activity on every cycle, whereas in the benchmark a table is written once per block and read one entry per symbol; an SRAM version would pay per access, not per cycle. Coverage percentages use the documented exclusions. Three MTF list invariants are proven for unbounded time by induction on a 16-entry list. At the production 256-entry width the general check is bounded to depth 6; a second run that starts from a valid filled list with 8 live entries shows that 24 consecutive moves preserve the permutation (hw/mtf_cam/synth/formal.sby). The regression counts are recorded results.

Does the hardware win?

Clocking and the link between the modules: both modules are single-clock synchronous designs and the chain uses **one shared clock; there is no clock-domain crossing**. The link is an AXI4-Stream valid/ready handshake carrying one symbol per beat. A shared clock runs at the slower module's frequency, 37.5 MHz, set by mtf_cam. We preferred this to two clocks joined by an asynchronous FIFO because the two estimates are 6% apart, so a crossing would add area, latency and verification burden for no benefit. The modules differ in cycles per symbol, not in clock: mtf_cam needs 1.0686 against the decoder's 1.0068, so it back-pressures the decoder through t_ready and the chain advances at mtf_cam's rate.

How it was tested: each side of the link is verified against the same beat-format contract and the same golden symbol stream, with back-pressure injected, and both handshakes are also checked formally. The two modules were then **simulated together** under one top level (hw/pyflate_accel) on the real benchmark block. The output is byte-exact over all 336,184 bytes, with and without random back-pressure on the output, in **159,303 cycles**, 0.5% above mtf_cam's standalone 158,441; the difference is the decoder's table-build start-up.

How the end-to-end time is estimated: offloading a stage removes its software time and adds the hardware's time plus the cost of moving data, $T_{\text{new}} = T_{\text{sw}} - T_{\text{stage}} + T_{\text{hw}} + T_{\text{if}}$. T_{sw} is the measured software run time and T_{stage} the measured time of the replaced stage. T_{hw} is the simulated chain cycle count divided by the clock frequency. T_{if} is the interface time: configuration, DMA setup and copies for the compressed input and the 336,184 L-vector bytes. It is not measured here, and transfers overlap compute only if buffers and bandwidth sustain it. The replaced stage was timed on its own: `dev/pyflate/phase_cpi.py` repeats the native decode phase 300 times after preparing its input and reports 3.304 ms per decode on the VM (`results/pyflate_phase_cpi.txt`; method and limits in Appendix A3).

Replaced stage only	Time	Evidence
Optimized Python loop	≈ 117 ms	derived: 283.88 – 170.01 + 3.30 ms
Rust kernel	3.30 ms	measured, Appendix A3
Hardware chain at 37.5 MHz (achievable)	≈ 4.25 ms	159,303 simulated cycles ÷ clock
Hardware chain at 50 MHz (design target)	≈ 3.19 ms	same cycles ÷ target clock

End to end	Time	vs original
Original Python	1,123.49 ms	1.00×
Optimized Python	281.16 ms	4.00×
Python + Rust kernel	170.01 ms	6.61×
Python + hardware chain at 37.5 MHz	≈ 171 ms	≈ 6.6× (projected)
Python + hardware chain at 50 MHz	≈ 170 ms	≈ 6.6× (projected)

The two hardware rows are projections and do not include the interface time T_{if} (configuration, DMA setup and copies), which is not measured; adding it can only make them slower.

Verdict: for the stage it replaces, the hardware chain is about 28× faster than the optimized Python loop and about 1.3× slower than the Rust kernel at the achievable clock, with parity at the 50 MHz target. End to end it ties the delivered 170 ms path, because once this stage is off the interpreter it is about 2% of what remains; inverse BWT (137.7 ms, about 80%) sets the floor for both the native and the hardware route. The cycle count is measured in simulation; the clock is a static-timing estimate and the interface time is unmeasured, so these are projections, not a run of Python attached to hardware. The 117 ms figure combines separate experiments and is approximate.

Why the symbol stage and not inverse BWT: after the Python optimizations the largest remaining stage is inverse BWT, so the choice needs a reason. The inverse transform is one serial pointer chase, `end = T[end]`, executed once per output byte over a 336,184-entry traversal table: every step depends on the previous load, so parallel arithmetic units gain nothing, and a hardware walker is bound by the latency of a random access into a table that does not fit a small on-chip memory (336 kB as bytes, 1.3 MB as Python int objects). About 336,000 dependent loads at main-memory latency is tens of milliseconds, the same order as the software it would replace. Table construction (the counting sort) is streaming and would accelerate well, but it is the smaller half (the traversal alone measured 47.8 ms against 137.7 ms for the whole transform, Appendix A3). The symbol stage, by contrast, streams through a 256-entry state, which is why both the Rust kernel and the accelerators take that boundary. A BWT engine would need an on-chip 336 kB SRAM for the traversal table, or a different transform; that is the next experiment, not this one.

Area and throughput trade-offs

Huffman storage includes a 17.3 kbit symbol table and an 8.6 kbit length window implemented as flops in this standard-cell flow. SRAM is a proposed alternative: the documented estimate is about 0.75 mm² of **remaining standard cells plus two macros**. Macro area, access latency, ports and integration must be established before claiming total area below 1 mm² or unchanged throughput. No SRAM version was synthesized here.

MTF output bytes/cycle	Mapped area	Modeled cycles/symbol
4-byte output	0.182 mm ²	1.175
8-byte output (selected)	0.187 mm ²	1.063
16-byte output	0.208 mm ²	1.023

$W = 8$ is the smallest modeled width meeting the 1.10 cycles/symbol goal; $W = 16$ costs 10.9% more area for about 3.9% modeled throughput gain. The W -sweep uses the cycle model; the default RTL's measured 1.0686 includes additional implementation overhead. The list's 68% area share explains why reducing packer width saves little total area. Timing must be checked for each implementation rather than assuming identical frequency across widths.

6. Conclusion

The delivered Python-plus-Rust path decompresses the input in **170.01 ms**, achieving **6.61× over the original** with byte-exact output. Python improvements provide 4.00× and the native decoder adds 1.67×, leaving BWT and RLE4 as the next software targets.

In hardware, `huffman_engine` and `mtf_cam` implement the same symbol-decode boundary as the Rust kernel. Simulated together on the real benchmark block they are byte-exact over all 336,184 output bytes and need 159,303 clock cycles, about 4.25 ms at the 37.5 MHz clock that static timing supports. That is about 28× faster than the optimized Python loop and about 1.3× slower than the Rust kernel (parity at the 50 MHz design target), for 1.8 mm² of 130 nm standard cells. End to end this ties the delivered 170 ms path: once the symbol stage leaves the interpreter it is about 2% of what remains, and inverse BWT, still in software, sets the floor for both routes. The hardware figures are projections from RTL simulation and static timing, not a run of Python attached to hardware.

Evidence: `hw/huffman_engine/docs/{ppa,integration}.md`, `hw/mtf_cam/docs/{ppa,integration}.md`, `hw/pyflate_accel/README.md`, `hw/docs/hardware_report.md`. Methods and profiles: **report_appendix.pdf**.